

BATPWN 2020 Crypto - Cryptographer

deuterium — 2020-06-08

Canonical HTML: <https://deut-erium.github.io/WriteUps/2020/batpwn/crypto/Cryptographer/2020-06-08-BATPWN-2020-Cryptographer.html>

Abstract

CTF writeup: BATPWN 2020's Cryptographer spells its timestamp XOR key in ASCII zeroes and ones. Each key byte has just two career options.

Cryptographer

Challenge

76 Solves



Cryptographer

100

We hired a new crypto analyst on 14th may for securing our systems but he used noob methods can you decode what was he trying to do ?

Author: bugluck

 noob.py

 noob.txt

Flag

Submit

We are provided with noob.txt which has it contents:-

```
U1FEQEdeS1JDSEBEXlZDUEFYSG5ZQ29TVVBFRFlXRFxvUUJFTA==
```

and noob.py which contents read

```
#!/usr/bin/env python2
# I AM NOOB :)
import string
from hashlib import md5
from itertools import izip, cycle
import base64
import time

def xor(data, key):
    return bytearray(a^b for a, b in zip(*map(bytearray, [data, key])))

flag=""

timestamp = time.time()
print int(timestamp)
key = md5(str(int(timestamp))).hexdigest()
my_hexdata = key

scale = 16
num_of_bits = 8
noobda = bin(int(my_hexdata, scale))[2:].zfill(num_of_bits)
print noobda
xorer(flag,noobda)
noobie = base64.encodestring(xorer).strip()
print noobie
```

Hmm, python2 🙄 and quite a wierd script.

Without using much of my small brain, I churned up a script quickly thinking I can bruteforce starting from current time backwards.

But it was actually xoring the flag with two values b'0' or b'1', So there are a very little number of possiblities to try.

```
import base64
from hashlib import md5
import time

def xor(data, key):
    return bytearray(a^b for a, b in zip(*map(bytearray, [data, key])))

timestamp_initial = int(time.time())
with open('noob.txt', 'r') as ct_file:
    ct = ct_file.read().strip()

ct = base64.b64decode(ct)

for i in range(1000000):
    timestamp = timestamp_initial - i
    key = md5(str(int(timestamp))).hexdigest()
    my_hexdata = key
    scale = 16
    num_of_bits = 8
    noobda = bin(int(my_hexdata, scale))[2:].zfill(num_of_bits)
```

```
xorer = xor(ct, noobda)
if 'batpwn{' in xorer and '}' in xorer:
    print(xorer)
```

```
batpwn{bsxptogr`qhy^is_cd`uugltl^ast}
batpwn{crxpunfs`phx_ir^bd`uthgum_art}
batpwn{crypuogr`qhx^is_ceatuifum^aru}
batpwn{bsxquogs`phy^hr^ceauthgul_asu}
batpwn{csypungrapix_hs_cd`tuigtlt^asu}
batpwn{csxqtnfr`qix^hs^bdauthgtl^`st}
batpwn{bsyqtnfs`qix^hr_bd`uthfum^ast}
batpwn{brxquogr`qiy_hs_ceatuigtm^`st}
batpwn{bsxqtogr`qhy_hs_cdatuhgtm^aru}
batpwn{csyqungsaphx_is_cdauuhftm^`rt}
batpwn{brxqungr`qix^hr_ceauthful_aru}
batpwn{csyqtngr`phy^is_cd`tuhfum_aru}
batpwn{brxquograqix_hs^bdautigtm`rt}
batpwn{bsxqtotsaqiy^hr_ceauuhftl`ru}
batpwn{csxqtotsaqhy^hs^beatuhgum_ast}
batpwn{brxqunfsaqhy_is_cd`uuifum^`rt}
batpwn{brxqunfsaphx_hr_be`ttiftl_asu}
batpwn{cryqunfsaqiy_hr^ceautiful^`rt}
batpwn{brxqtngr`pix^ir_ce`utigul`ru}
batpwn{csyqunfsaqhx^is_cdauuiftm^ast}
batpwn{bryptnfsaqhx^hr_beautigtm_asu}
batpwn{crxpuofr`phx_hr_cdautigul_aru}
batpwn{crxqungs`qhy^hr_ce`tuiful_ast}
batpwn{bryqungs`qhx_is_cd`tuhgtl_aru}
batpwn{bryqtnfr`qiy_ir_bd`tuigtlt_aru}
batpwn{csxptofr`qiy^is^bdathgtl^`st}
batpwn{bsypunfrapix_hr^bd`uthgum`st}
batpwn{bryptngs`qhy^hr_cdauthgtm_aru}
batpwn{csypunfsaphx_is_be`uuhgul^ast}
batpwn{csyqtogr`qhx^hr^cd`utigtm`st}
batpwn{bsxqtofr`qiy^hs^ce`uthfum^`rt}
batpwn{csyqtotsaqix_hs_be`uthftl`st}
batpwn{crxpunfrapiy^hr^beauthgtm_ast}
batpwn{brxptogsapix^ir_be`uthgum^aru}
batpwn{bsxptngs`qix^hr_ceatuiftl_aru}
batpwn{bsyqtogsaqhy^ir_cd`uuhful`st}
batpwn{csxqtngs`qhx^is_cdauthgum_aru}
batpwn{brxqtograpiy^ir_ce`ttiftm`su}
batpwn{bryptofr`qix_ir^cdatthgum`ru}
```

HAHAHA, without using much brain, one could tell that flag is
batpwn{cryptography_is_beautiful_art}

Interactive controls are omitted from this PDF. See the canonical HTML version.